

NLP - PRETRAINING

# Complete Exam Notes

35 Concepts - 7 Parts - Story-Style - Exam-Ready

Pre-Training - Fine-Tuning - BERT Family - Retrieval Models - Architectures - Post-Training -  
Prompting

Built for subjective exam answers - 4-5 lines, AI-checked, keyword-rich

Each concept: Story - Key Ideas - Pros/Cons - Exam Tip - Bridge to Next

# The Foundation

Core ideas everything builds upon

1. Pre-Training - Why and What
2. Fine-Tuning
3. Subword Modeling + BPE
4. Old Approach: Word Embeddings
5. Modern Approach: Whole Model Pretraining
6. Language Modeling (Autoregressive)

# Pre-Training

## The Foundation of Modern NLP

Imagine hiring someone who has never read anything - every task means learning from zero. That was early NLP: one model, one task, trained from scratch every time - slow, expensive, poor. The breakthrough: **make the model learn general language intelligence first on massive unlabeled data, then adapt it to specific tasks**. The model creates its own labels by predicting masked/next words - this is **Self-Supervised Learning (SSL)**. No human annotation needed, just raw internet text.

### X Problem with Old Approach

- Train model from scratch for every new task - very slow
- Needs huge labeled data - expensive, slow, limited scope
- Poor performance on low-resource tasks
- No knowledge transfer between tasks at all

### OK Solution

Pre-train on billions of words (no labels). Model learns grammar, context and world knowledge via SSL.

**Why next?** Pre-trained model has general knowledge but doesn't know your task - needs Fine-Tuning to adapt.

### \* What is Pre-Training?

#### ■ One-liner

Make model learn general things **before** giving specific tasks.

#### ■ Training type

**Self-Supervised Learning (SSL)** - model creates its own labels.

#### ■ Data used

Massive unlabeled text (internet, books) - no human annotation.

#### ■ Objective

Predict missing or next words - forces deep language understanding.

#### ■ Scale

Compute-aware: balance Data + Model size + GPU/TPU power.

### + What the Model Learns

#### ■ Grammar

Correct sentence structure without explicit grammar rules.

#### ■ Context

Same word - **different meaning** in different sentences (e.g., 'bank').

#### ■ World knowledge

Facts about people, places, events seen in training text.

#### ■ Reasoning

Patterns of logical inference from large corpora.

#### ■ Word relations

Synonyms, antonyms, analogies - all implicitly encoded.

### + Pros

#### ■ Less labeled data

Fine-tuning needs hundreds of examples, not millions.

#### ■ Faster training

Task-specific training converges quickly from good init.

#### ■ Better performance

SOTA on almost every NLP benchmark after pretraining.

#### ■ Generalization

One pretrained model - many tasks (QA, NER, sentiment).

### - Cons

#### ■ Expensive pretrain

Requires TPUs, weeks of training - not for everyone.

#### ■ Static knowledge

Doesn't know events after training cutoff date.

#### ■ Large storage

Pretrained models are huge (hundreds of millions of params).

#### ■ Compute-aware limit

Must balance data, model, compute or training fails/slows.

**TIP Exam Tip: WHY pretraining: avoids scratch training, no labeled data, learns grammar+context+world knowledge. HOW: SSL, predict next or masked word, compute-aware scaling. Key term: Self-Supervised Learning.**

### LINK Bridge to Next Concept

Pre-training gives general intelligence but the model doesn't know your specific task. **Solution - Fine-Tuning:** slightly adjust the pretrained weights on a small labeled dataset. Pretrain once, fine-tune many times - this is the paradigm that changed NLP.

# Fine-Tuning

## Adapting Pretrained Intelligence to Specific Tasks

The pre-trained model is a university graduate who knows everything in general. Fine-tuning is the residency/specialization: **take pretrained weights and slightly adjust them on a small task-specific labeled dataset**. Because the model already understands language deeply, it needs very little task-specific data. The magic of the **Pretrain - Fine-tune paradigm**: pretrain once on huge data (expensive), then fine-tune cheaply many times for different tasks.

### \* What is Fine-Tuning?

#### ■ Definition

**Slightly adjust** pretrained model weights for a specific task.

#### ■ How

Load pretrained model - train on small labeled task dataset.

#### ■ Data needed

Hundreds to thousands of examples (vs millions for pretraining).

#### ■ Key idea

Model already knows language; just needs task-specific context.

#### ■ Output

Task-specific model (spam detector, QA system, chatbot, etc.).

### = Pretrain vs Fine-tune

#### ■ Pretraining cost

**Very expensive** - TPUs, huge data, days/weeks.

#### ■ Fine-tuning cost

**Cheap and practical** - single GPU, hours.

#### ■ Key principle

**Pretrain once, fine-tune many times** for different tasks.

#### ■ Param init

Pretraining gives good starting weights - better than random init.

#### ■ Transfer

Knowledge learned in pretraining is reused - this is Transfer Learning.

## \* How Fine-Tuning Works

### ■ Step 1

Load pretrained model (e.g., BERT, GPT) with all its learned weights.

### ■ Step 2

Add a small **task-specific layer** on top (e.g., classification head).

### ■ Step 3

Train on small labeled dataset - **all weights update slightly**.

### ■ Step 4

Model adapts its general language knowledge to the specific task.

### ■ Examples

Spam detection, Sentiment analysis, QA, Named Entity Recognition, Translation.

### ■ One-line

BERT is pretrained using MLM then fine-tuned for tasks - this is **Transfer Learning**.

**TIP Exam Tip: Fine-tuning = take pretrained model + slightly adjust on small labeled dataset. Advantages: less data, faster convergence, better performance. Key: pretrain once, fine-tune many times.**

## LINK Bridge to Next Concept

Before feeding text to any model, we must convert words to tokens. Old NLP had a fatal flaw: **fixed vocabulary** - any new/rare word became [UNK] (unknown), losing all meaning. The solution is **Subword Modeling + BPE**: break words into smaller meaningful pieces.

# Subword Modeling + BPE

## Solving the Unknown Word Problem

Old NLP models assumed a fixed vocabulary of 30k-50k words. Any word not in this list - a typo, a new brand name, a rare scientific term - got mapped to **[UNK] (unknown)**, losing all meaning. The insight: **words are combinations of smaller units**. 'Transformerify' = 'Transformer' + 'ify'. Even unseen words can be represented using known subword pieces. **Byte Pair Encoding (BPE)** is the algorithm that learns these splits automatically from data.

### X Problem with Old Approach

- NLP models assume fixed vocabulary (e.g., 30k-50k words)
- New/rare words - mapped to [UNK] (unknown token) - loss of meaning
- Cannot handle: misspellings ('laern'), variations ('taaasty'), new words ('Transformerify')
- Fixed vocabulary is not suitable for real-world language

### OK Solution

Break words into **subwords (parts of words)**. Word = combination of smaller units. Common words kept whole, rare words split into known pieces.

**Why next?** With subwords handled, text can be properly tokenized. Now how do we represent these tokens? Old approach used static word embeddings - which had a major flaw.

### # BPE Algorithm

#### ■ Full form

Byte Pair Encoding - algorithm to **learn subwords from data**.

#### ■ Step 1

Start with individual **characters** as the vocabulary.

#### ■ Step 2

Find the **most frequent adjacent pair** of tokens.

#### ■ Step 3

**Merge** that pair into a single new token.

#### ■ Step 4

Repeat until vocabulary size is reached (e.g., 30k-50k tokens).

### ~ Subword Examples

#### ■ Common word

hat - **hat** (kept whole - frequent enough).

#### ■ Variation

taaasty - **taa + aaa + sty**

#### ■ Misspelling

laern - **la + ern** (handled gracefully, not UNK).

#### ■ New word

Transformerify - **Transformer + ify**

#### ■ '##' meaning

**Indicates continuation of a word** (e.g., ##ing, ##ify).

## ! Key Insights

### ■ Modern NLP = Subwords + Context

Subwords solve vocabulary, context (BERT/GPT) solves meaning.

### ■ No UNK needed

Any word can be decomposed into known subword pieces - nothing is unknown.

### ■ Scalable

Vocabulary stays manageable (30k-50k) but covers all possible words.

### ■ Used in BERT

BERT uses WordPiece (similar to BPE). GPT uses BPE (~40k tokens).

**TIP Exam Tip: Problem: fixed vocab - UNK. Solution: BPE breaks words into subwords. BPE steps: start with chars - find frequent pair - merge - repeat. '##' = word continuation.**

## LINK Bridge to Next Concept

With tokenization solved, how do we turn tokens into meaning? Old approach: pretrain only **word embeddings** (word2vec/GloVe), train rest from scratch. This had a fatal flaw - let's see why embedding-only pretraining was insufficient.



# Old Approach: Pretrained Word Embeddings

## Why Static Embeddings Were Insufficient

Before modern pretraining, the community had a partial solution: pre-train **word embeddings** (word2vec, GloVe) on large text, then use these as inputs to an LSTM/Transformer trained from scratch. This was better than random initialization but had a fatal flaw: **embeddings were static** - the word 'bank' had the same vector whether you meant river bank or money bank. The model had to learn everything else (grammar, context, reasoning) from the tiny task dataset.

### \* What Was Done

#### ■ Pretrained part

Only **word embeddings** (word2vec, GloVe) pretrained on large text.

#### ■ Rest of model

LSTM/Transformer trained **from scratch** on the task data.

#### ■ Benefit

Slightly better than fully random initialization.

#### ■ Used for

Only input layer initialization - rest was random.

### - Problems

#### ■ No context

Same word - **same vector always**. 'movie (good vs bad)' - same embedding.

#### ■ Heavy task burden

Small dataset must teach grammar, context, meaning from scratch.

#### ■ Random init

Most model parameters start random - very inefficient learning.

#### ■ Takeaway

**Pretraining only embeddings is insufficient.**

### ? The Context Problem - Why Static Fails

#### ■ Polysemy problem

'Bank' in 'river bank' vs 'money bank' - SAME embedding. Model can't distinguish.

#### ■ Static nature

GloVe gives ONE fixed vector per word - ignores all surrounding context.

#### ■ Small dataset burden

Task dataset (e.g., 10k sentences) must teach everything the model needs.

#### ■ Solution needed

Must pretrain the **entire model**, not just embeddings - Modern Approach.

**TIP Exam Tip:** Old approach: pretrain embeddings (word2vec/GloVe), model trained from scratch. Problems: no context (same vector always), heavy task dependency, random init for rest. Key takeaway: pretraining only embeddings is insufficient.

### LINK Bridge to Next Concept

The fix: instead of just pre-training embeddings, **pretrain the entire model** - encoder, attention layers, everything. Use a self-supervised objective: **hide parts of the input and train the model to reconstruct them**. This is the Modern Approach and a major breakthrough in NLP.

# Modern Approach: Pretraining Whole Models

The Breakthrough - Train Everything, Not Just Embeddings

Instead of just pre-training word vectors, the modern approach **pre-trains the entire model** - all layers, attention heads, and parameters - using a self-supervised objective: **hide parts of the input, train the model to reconstruct the missing parts**. Example: 'The movie was [MASK] - good' forces the model to understand full sentence context. This forces deep understanding at every level. **This is a major breakthrough in NLP.**

## > What Changed

### ■ Old

Pretrain embeddings only, model trained from scratch.

### ■ New

Pretrain **entire model** - all layers and parameters.

### ■ How

Hide parts of input - train model to reconstruct missing parts.

### ■ Example

'The movie was [MASK] - good'

### ■ Takeaway

Pretraining full model = **major breakthrough in NLP.**

## + What We Gain

### ■ Strong understanding

Model understands grammar, context, meaning at deep level.

### ■ Better param init

ALL parameters well-initialized - nothing random.

### ■ Language generation

Ability to model language probabilities and generate text.

### ■ Big insight

Solving fill-in-the-blank teaches: grammar, context, world knowledge, reasoning.

## @ Connection to Real Models

### ■ BERT

Uses **Masked Language Modeling (MLM)** - fill in the blanks - bidirectional context.

### ■ GPT

Uses **Next word prediction** - autoregressive - left-to-right only.

### ■ Why SSL?

No labels needed. The mask/position IS the label. Scales to any amount of text.

### ■ SSL advantage

No labeled data required. Uses huge internet text. Easy to scale indefinitely.

**TIP Exam Tip:** Modern pretraining = pretrain entire model (not just embeddings). Two main objectives: BERT = fill blanks (MLM), GPT = predict next word. Result: strong language understanding + better param init.

### LINK Bridge to Next Concept

The two main pretraining objectives split into two model families. First, let's understand the simpler one - **Language Modeling**: predict the next word. This is how GPT-style decoder models are pretrained and it scales beautifully to massive datasets with no labels needed.

# Language Modeling (Autoregressive)

Training by Predicting the Next Word - Foundation of GPT

The simplest and most powerful pretraining objective: given a sequence of words, **predict the next word**. Sounds trivial - but to predict the next word correctly, the model must understand grammar, context, sentence structure, and world knowledge. And crucially, **every sentence on the internet is already labeled** - the next word IS the label. No human annotation needed at all. This is called **Autoregressive Language Modeling**, the foundation of the GPT family.

## \* Core Idea

### ■ Task

Train model to **predict next word** given all previous words.

### ■ Name

This is called **Language Modeling**.

### ■ Math

$$p(w_t | w_1, w_2, \dots, w_{t-1})$$

### ■ Meaning

Probability of next word based on all past context.

### ■ Key term

**Autoregressive modeling** = predict next word step by step.

## ! Why This is Useful

### ■ No labeled data

The next word IS the label - completely free.

### ■ Huge text data

Uses all internet text - billions of sentences.

### ■ Easy to scale

More data - better model, no annotation bottleneck.

### ■ Natural

Predicting next word is natural language behavior.

## + What Model Learns

### ■ Grammar

Must produce grammatically correct next words.

### ■ Context

Must understand full sentence context to predict well.

### ■ Word relationships

Learns which words go together in what contexts.

### ■ Sentence structure

Implicitly learns syntax and semantics.

### ■ World facts

Learns factual associations from training text.

## = Model Type and Example

### ■ Type

**Decoder-based** models (GPT, LSTM older).

### ■ Direction

Left-to-right only - cannot see future tokens.

### ■ Example step 1

'Iroh goes -> to'

### ■ Example step 2

'Iroh goes to -> make'

### ■ Example step 3

'Iroh goes to make -> tasty'

**TIP Exam Tip:** Language Modeling = predict next word given past words. Math:  $p(w_t | w_1 \dots w_{t-1})$ . Type: autoregressive, decoder-based (GPT). Why useful: no labels, internet text, scalable. Model learns grammar+context+facts automatically.

### LINK Bridge to Next Concept

Language Modeling is left-to-right only. What if we could use **both left AND right context** to understand each word? That requires a different objective: **Masked Language Modeling (MLM)** - used by BERT. BERT is the most influential NLP model ever built. Let's dig in.

# The BERT Family

## Bidirectional Encoders and Their Extensions

1. BERT - Core Architecture, MLM and Transfer Learning
2. BERT - Input Format, Embeddings and NSP
3. BERT - Training Objectives (Deep Dive)
4. BERT - Fine-tuning on Different Tasks
5. Contextualized Representations: ELMo vs GloVe
6. Limitations of Pretrained Encoders
7. RoBERTa + SpanBERT - BERT Extensions
8. ELECTRA - Replaced Token Detection
9. ALBERT + DistilBERT - Compression Methods
10. Longformer - Efficient Attention for Long Docs

# BERT

## Bidirectional Encoder Representations from Transformers

BERT changed NLP forever. Before BERT, models could only see context from one direction (left-to-right). BERT's key insight: **read both left and right context simultaneously** - just like humans read. The word 'bank' in 'river bank' and 'money bank' gets different representations because BERT sees the full sentence from both sides. Pretrained using **Masked Language Modeling (MLM)** on Wikipedia + BooksCorpus, BERT became the starting point for almost every NLP task.

### \* BERT Overview

#### ■ Full form

Bidirectional Encoder Representations from Transformers

#### ■ What's special

Reads sentence from **left + right simultaneously** (bidirectional).

#### ■ Pretraining

Uses **Masked Language Modeling (MLM)** to learn bidirectional context.

#### ■ Data

BooksCorpus (~800M words) + Wikipedia (~2.5B words).

#### ■ Variants

BERT-base: 12 layers, 110M params.  
BERT-large: 24 layers, 340M params.

### ~ Masked Language Modeling (MLM)

#### ■ What

Randomly select **15% of tokens** for prediction.

#### ■ 80% of masked

Replace with **[MASK]** token.

#### ■ 10% of masked

Replace with a **random word**.

#### ■ 10% of masked

**Keep unchanged** (but still predict it).

#### ■ Why strategy?

Prevents model relying only on [MASK]. Makes robust to real-world inputs.

### @ Transfer Learning with BERT

#### ■ Transfer Learning

Knowledge learned in pretraining is **reused** for new tasks - no training from scratch.

#### ■ Advantages

Less labeled data required. Faster training. Better performance. Better generalization.

#### ■ One-line takeaway

BERT is pretrained using MLM and then fine-tuned for tasks - this is Transfer Learning.

#### ■ What BERT learns

Word meaning (MLM), Context (bidirectional), Word relationships, Grammar.

#### ■ Parameter init

Pretraining gives good starting weights - much better than random initialization.



**TIP Exam Tip: BERT = Bidirectional Encoder Representations from Transformers. Bidirectional = reads left+right context. Pretraining = MLM (15% masked: 80% [MASK], 10% random, 10% unchanged). Why 80/10/10 strategy: prevents relying only on [MASK] token.**

### LINK Bridge to Next Concept

BERT's input format is very specific - it takes two sentences with special tokens and three types of embeddings. Understanding the input format and the NSP task is essential for exam questions. Let's go deep.

# BERT: Input Format + NSP

## Three Embeddings + Next Sentence Prediction

BERT doesn't just take a sentence - it takes **two chunks of text together** with special tokens that signal structure. Each token's final representation is the SUM of THREE embeddings: Token (what word is it?), Segment (which sentence does it belong to?), Position (where in the sequence is it?). This rich input encoding lets BERT understand both within-sentence meaning and between-sentence relationships.

### ! BERT Input Format (VERY IMPORTANT)

#### ■ Input

BERT takes **two sentences** as input simultaneously.

#### ■ Format

[CLS] Sentence A [SEP] Sentence B [SEP]

#### ■ Example

[CLS] My dog is cute [SEP] He likes playing [SEP]

#### ■ [CLS] token

Used for **classification tasks** - captures overall sentence meaning.

#### ■ [SEP] token

**Separates** the two sentences; signals sentence boundaries.

### # Three Types of Embeddings (VERY IMPORTANT)

#### ■ (1) Token Embedding

**Meaning of the word** itself - what is this token?

#### ■ (2) Segment Embedding

Which sentence does this token belong to? Sentence A - EA, Sentence B - EB.

#### ■ (3) Position Embedding

Position of word in sequence (0,1,2,...). Transformer has no order awareness by default.

#### ■ Final Embedding

**Token + Segment + Position** - all three summed together.

#### ■ Why position?

Transformer processes all tokens in parallel, so position must be explicitly added.

## @ Next Sentence Prediction (NSP)

### ■ Task

Given Sentence A and Sentence B - does B follow A?

### ■ Output

IsNext or NotNext

### ■ Prediction

Uses **[CLS] token representation** for binary prediction.

### ■ Example IsNext

A: 'I went to the store' - B: 'I bought milk' - IsNext

### ■ Example NotNext

A: 'I went to the store' - B: 'Penguins can fly' - NotNext

### ■ Important Note

NSP was later found **less useful**. Many modern models (RoBERTa) remove it.

**TIP Exam Tip:** BERT input = **[CLS] Sentence A [SEP] Sentence B [SEP]**. Three embeddings: **Token** (word meaning) + **Segment** (which sentence) + **Position** (order). NSP task: given two sentences, predict if B follows A. **[CLS]** used for classification.

## LINK Bridge to Next Concept

Now we understand BERT's input. The training objectives in detail - how exactly MLM and NSP work at the prediction level - is where most exam questions go. Let's deep dive into BERT's training objectives.

# BERT: Training Objectives

MLM + NSP - Deep Dive into How BERT Actually Learns

BERT learns from two simultaneously trained objectives. The first, **MLM**, teaches BERT to understand each word using full bidirectional context. The second, **NSP**, teaches BERT to understand relationships between sentences. Together they give BERT both word-level and sentence-level understanding - making it powerful for virtually any NLP task.

## ~ Objective 1: Masked Language Modeling (MLM)

### ■ Step 1

Randomly select **15% of input tokens** for masking.

### ■ Replacement

80% - [MASK], 10% - random word, 10% - unchanged.

### ■ Prediction

Sentence passes through **Transformer Encoder** - each token gets contextual representation.

### ■ For masked token

Take its hidden vector - pass through FFNN + Softmax - get probability over vocabulary.

### ■ Loss

Compare prediction with actual word - compute loss - update weights.

### ■ Key advantage

Model predicts using **full bidirectional context** (left + right).

## @ Objective 2: Next Sentence Prediction (NSP)

### ■ Input

Two sentences: A and B.

### ■ Task

Check if sentence B actually follows sentence A in the original document.

### ■ Outputs

IsNext or NotNext

### ■ Prediction token

Uses **[CLS] token representation** for binary classification.

### ■ What BERT learns

Word meaning (MLM) + Context (MLM) + Sentence relationships (NSP).

### ■ Note

NSP later found less useful - many modern models drop it entirely.

## ! What BERT Learns Overall

### ■ From MLM

Word meaning, Grammar, Context - using bidirectional attention.

### ■ From NSP

Sentence relationships, discourse structure.

### ■ Deep representations

Learns **deep contextual representations** of words - context-aware.

### ■ Bidirectional

Uses context from **both left AND right** - unique to BERT vs GPT.

**TIP Exam Tip: BERT trains with MLM (15% tokens: 80% [MASK], 10% random, 10% unchanged) + NSP. MLM: sentence - Transformer encoder - contextual rep - FFNN+Softmax - predict masked word. NSP: IsNext or NotNext using [CLS] token. NSP later found less useful.**

## LINK Bridge to Next Concept

With BERT trained, it can be fine-tuned for many tasks. How BERT adapts to different task types - classification, sequence labeling, QA - is a key exam topic. Each task uses a different part of BERT's output.

# BERT: Fine-tuning on Different Tasks

How BERT Adapts to Any NLP Task

BERT is a pretrained transformer encoder that can be fine-tuned for multiple NLP tasks by adding small task-specific layers on top. The key is **which part of BERT's output you use** for each task type. Classification tasks use the [CLS] token, sequence tasks use all token outputs. BERT achieves state-of-the-art results on the GLUE benchmark across all these tasks.

## \* 1. Sentence Classification

### ■ Input

Single sentence.

### ■ Output

One label for the entire sentence.

### ■ Uses

[CLS] token representation for prediction.

### ■ Examples

Sentiment analysis (positive/negative),  
Acceptability (CoLA).

## @ 2. Sentence Pair Classification

### ■ Input

Two sentences separated by [SEP] token.

### ■ Output

Relationship between the two sentences.

### ■ Examples

Entailment (MNLI, RTE), Paraphrase detection  
(MRPC, QQP), Semantic similarity (STS-B).

## # 3. Sequence Labeling (Token-level)

### ■ Input

Sentence.

### ■ Output

Label for each word/token.

### ■ Examples

Named Entity Recognition (NER), POS  
Tagging.

### ■ How

Use output representation of EACH token (not  
just [CLS]).

## ? 4. Question Answering (MRC)

### ■ Input

Question + Passage.

### ■ Output

Start and End positions of answer in passage.

### ■ Example dataset

SQuAD.

### ■ How

Predict start token and end token of answer  
span.

## ! 5. Performance - GLUE Benchmark

### ■ GLUE

General Language Understanding Evaluation - multiple NLP tasks in one benchmark.

### ■ BERT result

Achieves **state-of-the-art results** on multiple NLP tasks simultaneously.

### ■ Key insight

Shows **strong general language understanding** ability from pretraining.

### ■ Takeaway

One pretrained BERT model fine-tuned separately for each task - SOTA everywhere.

**TIP Exam Tip: BERT on tasks: Sentence classification - [CLS] token. Sequence labeling - all tokens. QA - start+end positions in passage. Sentence pair - [SEP] between sentences. BERT achieves SOTA on GLUE benchmark.**

### LINK Bridge to Next Concept

Static word embeddings (GloVe) gave one fixed vector per word. BERT gives different vectors for the same word in different contexts. This is **Contextualized Representations** - a fundamental difference worth examining closely.

# Contextualized Representations

## ELMo vs GloVe - Static vs Dynamic Embeddings

The word 'bank' has completely different meanings in 'river bank' and 'money bank'. **Static embeddings (GloVe)** give it the same vector regardless of context. **Contextual embeddings (ELMo/BERT)** generate a DIFFERENT vector for each occurrence based on the surrounding sentence. This is the key difference: contextual models understand that the same word can mean different things in different places.

### - GloVe - Static (Old)

#### ■ Type

Static - one fixed vector per word.

#### ■ Same word

'bank' - SAME embedding always, regardless of context.

#### ■ Ignores context

Surrounding words have no effect on the embedding.

#### ■ Problem

Cannot handle polysemy (words with multiple meanings).

#### ■ How trained

Trained on word co-occurrence statistics - not context.

### + ELMo/BERT - Contextual (Modern)

#### ■ Type

Contextual - different vector for same word in different contexts.

#### ■ Same word

'bank' - different embedding in 'river bank' vs 'money bank'.

#### ■ Uses context

Each word's vector depends on the **entire sentence**.

#### ■ How generated

Generated on-the-fly by passing sentence through model.

#### ■ Pretrained models

Provide context vectors for each word dynamically.

## = ELMo vs GloVe Summary

#### ■ GloVe (Static)

One fixed vector per word. Ignores context. Simple but limited.

#### ■ ELMo (Contextual)

Different vector for same word in different sentences. Context-aware.

#### ■ BERT (Contextual)

Like ELMo but bidirectional Transformer - much deeper context understanding.

#### ■ Key difference

Unlike traditional methods, contextual embeddings are NOT fixed - generated on-the-fly.

#### ■ Why it matters

Polysemy handled correctly. Word meaning changes based on full sentence context.



**TIP Exam Tip:** GloVe = static, one vector per word, ignores context. ELMo/BERT = contextual, different vector for same word in different sentences, generated on-the-fly. Key: each word's vector depends on the entire sentence.

### LINK Bridge to Next Concept

BERT is powerful for understanding tasks - but it has a critical limitation: it **cannot generate text sequentially**. Understanding this limitation is important because it explains why we need decoder models (GPT) for generation tasks.

# Limitations of Pretrained Encoders (BERT)

## Why BERT Cannot Generate Text

BERT is extremely powerful for **understanding** tasks - classification, QA, NER - but it has a fundamental limitation: **it cannot naturally generate text**. Why? Because BERT was trained with MLM - it predicts missing words using FULL bidirectional context (left + right). Real text generation requires producing words one by one, left-to-right, without seeing future tokens. BERT fundamentally cannot do this autoregressive generation.

### + What BERT CAN Do

#### ■ Understanding tasks

Classification, QA, NER - anything requiring understanding.

#### ■ Bidirectional context

Uses full left + right context for predictions.

#### ■ GLUE tasks

Entailment, sentiment, paraphrase, similarity.

#### ■ Extraction

Extractive QA - find answer span in passage.

### - What BERT CANNOT Do

#### ■ Text generation

Cannot generate new text word by word.

#### ■ Autoregressive

Cannot do sequential word-by-word output.

#### ■ Translation

Cannot translate sentences (requires generation).

#### ■ Summarization

Cannot write summaries (requires generation).

## ? Why BERT Cannot Generate - The Core Reason

#### ■ MLM training

BERT predicts missing words using **full context (left + right)**.

#### ■ Not sequential

It does NOT generate text sequentially word by word.

#### ■ Bidirectional = problem

Sees future tokens during training - cannot mask them during generation.

#### ■ Autoregressive fail

Cannot naturally perform word-by-word output generation.

#### ■ Solution

For generation tasks (text gen, translation, chat): use **pretrained decoders (GPT)**.

#### ■ Key takeaway

**Encoders (BERT) - Understanding tasks. Decoders (GPT) - Generation tasks.**

**TIP Exam Tip: BERT limitations: powerful for understanding (classification, QA, NER) but NOT for text generation. Reason: trained with MLM using full bidirectional context - cannot do sequential word-by-word output. Key: Encoders - Understanding. Decoders - Generation.**

### **LINK Bridge to Next Concept**

Despite these limitations, BERT's architecture is excellent. Researchers found ways to make BERT even better by improving the training recipe (RoBERTa) and changing the masking strategy (SpanBERT). Let's look at these BERT extensions.

# RoBERTa + SpanBERT

BERT Extensions - Better Training + Span-level Understanding

After BERT, researchers found two key ways to improve it. **RoBERTa** showed that BERT was significantly undertrained - simply training longer with more data and removing NSP gave dramatically better results. **SpanBERT** changed WHAT gets masked - instead of random words, mask entire continuous spans of text, because language meaning is often carried in phrases, not isolated words.

## > RoBERTa - Better Training Recipe

### ■ Same architecture

Same as BERT - not a new model design.

### ■ Improvement 1

**Train longer** - more training steps.

### ■ Improvement 2

**Use more data** - performance improves.

### ■ Improvement 3

**Remove NSP** - found less useful, hurts performance.

### ■ Result

Training MUCH longer - best results achieved.

### ■ Key message

**Not a new model - just better training.**  
Training recipe matters!

## ~ SpanBERT - Span-level Masking

### ■ Problem with BERT

BERT masks random individual words - but phrases carry meaning.

### ■ Solution

Mask **contiguous spans (chunks) of words** instead of random words.

### ■ BERT example

It [MASK] irre [MASK] sible [MASK] [MASK]  
good

### ■ SpanBERT example

It [MASK] [MASK] [MASK] [MASK] [MASK]  
good

### ■ Why better?

Language understanding often depends on phrases, not single words.

## ! SpanBERT - Span Boundary Objective (SBO)

### ■ Key idea

Use only the **left and right boundary tokens** of a masked span to predict it.

### ■ Boundary tokens

These boundary representations used to predict the **whole span content**.

### ■ Motivation

Natural language meaning is often captured in phrases, not isolated words.

### ■ Better at

**Phrase-level understanding**, QA, NER, Coreference Resolution, Relation Extraction.

### ■ Takeaway

Predicting entire spans leads to better contextual and semantic understanding.

**TIP Exam Tip: RoBERTa = same as BERT but: train longer, more data, no NSP. NOT a new model - just better training. SpanBERT = mask continuous spans (not random words) + Span Boundary Objective. Better for phrase-level tasks: QA, NER, coreference.**

### LINK Bridge to Next Concept

Both RoBERTa and SpanBERT still use BERT's MLM objective - they improve **what** and **how much** is masked. ELECTRA takes a completely different approach: instead of predicting masked words, it uses a **Generator-Discriminator architecture** that learns from ALL tokens, not just 15%.

# ELECTRA

Efficiently Learning an Encoder - Replaced Token Detection

BERT's MLM has a fundamental inefficiency: it only learns from the 15% of tokens it masks. The other 85% provide no learning signal. ELECTRA replaces this with a far more efficient objective: **Replaced Token Detection (RTD)**. A small Generator model (like BERT) fills in masked tokens, then the main Discriminator model must identify which tokens are 'real' (original) and which are 'fake' (generated by the Generator). This happens for **EVERY token - 100%**.

## \* ELECTRA Architecture

### ■ Core idea

Replace MLM with **Replaced Token Detection (RTD)**.

### ■ 1. Generator (small)

Similar to BERT MLM. Predicts masked tokens.  
Used only to create corrupted sentences.

### ■ 2. Discriminator (main)

Takes corrupted sentence. Predicts for EACH token: **real or fake**.

### ■ Final model

Only the Discriminator is used after training.

## = Training Process

### ■ Step 1

Mask some tokens in input.

### ■ Step 2

Generator predicts masked tokens (correct or incorrect).

### ■ Step 3

Replace masked tokens with Generator outputs.

### ■ Step 4

Discriminator checks EVERY token: real or fake?

### ■ Step 5

Compute loss and update both models.

## ! Why ELECTRA is Better than BERT

### ■ Uses ALL tokens

BERT learns from 15%. ELECTRA learns from **100% of tokens**.

### ■ More compute-efficient

Gets same performance as BERT with less training compute.

### ■ Stronger context

Learns whether a word **fits naturally in context** - deep grammar + semantics.

### ■ Faster training

Achieves better performance with **less training time**.

| Feature         | BERT        | ELECTRA                  |
|-----------------|-------------|--------------------------|
| Objective       | Masked LM   | Replaced Token Detection |
| Learning signal | ~15% tokens | All tokens (100%)        |
| Efficiency      | Lower       | Higher                   |
| Training speed  | Slower      | Faster                   |
| Performance     | Strong      | Stronger per compute     |

**TIP Exam Tip: ELECTRA = Generator (small, creates fake tokens) + Discriminator (main model, detects real vs fake). Objective: Replaced Token Detection on ALL tokens (vs BERT's 15%). Why better: uses 100% tokens, more compute-efficient, learns stronger contextual representations.**

### LINK Bridge to Next Concept

BERT, RoBERTa, SpanBERT, ELECTRA are all large models. What if we need a smaller, faster model while maintaining similar performance? Two approaches: **ALBERT** (reduce parameters via architectural tricks) and **DistilBERT** (Knowledge Distillation - train a small model to mimic a large one).

# ALBERT + DistilBERT

Compression Methods - Smaller, Faster, Still Powerful

BERT-base has 110M parameters and BERT-large has 340M - very expensive to deploy. **ALBERT** reduces parameters through smart architectural tricks without sacrificing much performance. **DistilBERT** uses **Knowledge Distillation**: train a small 'student' model to mimic the behavior of the large 'teacher' BERT. Both achieve the goal: smaller, faster, cheaper models that retain most of BERT's power.

## \* ALBERT - A Lite BERT

### ■ Goal

Reduce model size (parameters) while maintaining BERT-level performance.

### ■ Problem with BERT

Very large (110M-340M params). High memory and computational cost.

### ■ Technique 1

**Factorized Embedding Parameterization:** Split large embedding matrix into two smaller ones. Separates vocabulary size from hidden size.

### ■ Technique 2

**Cross-Layer Parameter Sharing:** Same parameters across ALL transformer layers (1 transformer used 12 times, instead of 12 separate transformers).

### ■ Result

BERT-base 110M - ALBERT-base ~12M. Maintains similar performance with much lower memory.

### ■ Key takeaway

Efficient parameter design can reduce model size without sacrificing performance.



## @ DistilBERT - Knowledge Distillation

### ■ Goal

Smaller, faster, cheaper model with similar performance to BERT.

### ■ Method

**Knowledge Distillation** - model compression technique.

### ■ Teacher

Large BERT model - the 'expert'.

### ■ Student

DistilBERT - small model that **mimics the teacher**.

### ■ How

Student learns from teacher's **output probabilities (soft targets)**, not just ground truth labels.

### ■ Why soft targets?

Teacher's probability distribution over all words captures more information than a one-hot label.

### ■ Advantages

Reduced model size, faster inference, lower compute, retains ~97% of BERT's performance.

### ■ Key idea

Train a smaller model to **imitate a larger pretrained model** instead of learning from scratch.

**TIP Exam Tip: ALBERT: two tricks - (1) Factorized Embedding (split big matrix into two small ones), (2) Cross-Layer Parameter Sharing (same params reused across all layers). Result: 110M - 12M params. DistilBERT: Knowledge Distillation - small student learns from large teacher's soft probability outputs.**

## LINK Bridge to Next Concept

ALBERT and DistilBERT address parameter efficiency. But BERT also struggles with **long documents** - standard Transformer attention is  $O(n^2)$ , making it impractical for documents with thousands of tokens. **Longformer** solves this with efficient attention patterns.

# Longformer

## Efficient Attention for Long Documents - 4 Attention Patterns

Standard Transformer attention has a critical problem: every token attends to every other token. With 1k tokens that's 1M interactions; with 10k tokens that's 100M interactions -  **$O(n^2)$  complexity**. This makes BERT practically unusable for long documents (legal texts, books, medical records). Longformer introduces four attention patterns that dramatically reduce this complexity while preserving the ability to capture long-range dependencies.

### - (a) Full Attention - Standard

#### ■ Every token

Attends to every other token.

#### ■ Complexity

$O(n^2)$  - worst efficiency.

#### ■ Best performance

Fully dense matrix - each word looks at every word.

#### ■ Problem

1k tokens - 1M interactions. 10k - 100M.  
Impractical for long docs.

### ~ (b) Sliding Window Attention

#### ■ Each token

Attends only to nearby tokens in a fixed window.

#### ■ Complexity

$O(n)$  - linear, much cheaper.

#### ■ Window

Word at position  $i$  attends to:  $i-2, i-1, i, i+1, i+2$ .

#### ■ Limitation

Cannot capture long-range dependencies directly.

### > (c) Dilated Window Attention

#### ■ Like sliding window

But with gaps (dilation) between attended tokens.

#### ■ Complexity

$O(n)$  - still linear.

#### ■ Example

Instead of neighbors, attends to  $i-2, i+2, i+4$ .

#### ■ Benefit

Wider local context without additional compute.

### + (d) Global + Sliding Window

#### ■ MOST IMPORTANT

This is Longformer's key idea.

#### ■ Combines

Local (sliding window) + Global attention for special tokens.

#### ■ Global tokens

[CLS] token and Question tokens attend to everything.

#### ■ Complexity

$O(n)$  - Local + global for few important tokens.

#### ■ Best trade-off

Local context everywhere + global context for key tokens.

| Method           | Complexity | Context        |
|------------------|------------|----------------|
| Full attention   | $O(n^2)$   | Full           |
| Sliding window   | $O(n)$     | Local          |
| Dilated window   | $O(n)$     | Wider local    |
| Global + sliding | $O(n)$     | Local + global |

**TIP Exam Tip:** BERT struggles with long docs because attention =  $O(n^2)$ . Longformer's 4 patterns: (a) Full  $O(n^2)$ , (b) Sliding Window  $O(n)$ , (c) Dilated Window  $O(n)$ , (d) Global+Sliding  $O(n)$  - BEST. Global tokens ([CLS], question tokens) attend to everything; all others use sliding window.

### LINK Bridge to Next Concept

BERT-family models store world knowledge **implicitly in model weights** - limited capacity, hard to update, poor interpretability. **REALM** introduces a radically different idea: instead of memorizing, the model **retrieves** information from an external corpus.

# Retrieval-Augmented Models

When Models Learn to Search Instead of Memorize

1. REALM - Retrieval-Augmented Pretraining
2. RAG - Retrieval-Augmented Generation

# REALM

## Retrieval-Augmented Language Model Pretraining

BERT and GPT store world knowledge **implicitly in model parameters (weights)**. This means limited memory capacity, hard-to-update knowledge, and poor interpretability. REALM proposes a paradigm shift: **retrieval-augmented pretraining**. Instead of memorizing everything, the model **retrieves relevant information** from an external corpus (e.g., Wikipedia) during both training and inference. The key idea: shift from **implicit knowledge (in weights)** - **explicit knowledge (via retrieval)**.

### X Problem with Old Approach

- BERT/GPT store world knowledge implicitly in model parameters (weights)
- Limited memory capacity - model can only memorize so much
- Hard to update knowledge - must retrain to add new facts
- Poor interpretability - can't see what knowledge is being used

### OK Solution

REALM proposes retrieval-augmented pretraining. Instead of memorizing, model retrieves relevant info from external corpus (Wikipedia). Shift: implicit knowledge - explicit knowledge via retrieval.

**Why next?** REALM retrieves but extracts short answer spans. RAG extends this to generate full, natural answers.

### = How REALM Works

#### ■ Key idea

Retrieve relevant documents - use as context - answer question.

#### ■ Pretraining

Model learns to retrieve AND use retrieved text simultaneously.

#### ■ During inference

Given question - retrieve from Wikipedia - generate answer.

#### ■ Benefits

No need to increase model size. Easier to update knowledge. Better on knowledge-intensive tasks.

### ? REALM vs BERT

#### ■ BERT

**Memorization** - facts stored in weights.

#### ■ REALM

**Search + use information** - retrieves from corpus.

#### ■ Knowledge update

BERT: retrain model. REALM: update the corpus.

#### ■ Interpretability

REALM: you can see what documents were retrieved.

#### ■ Performance

REALM outperforms BERT on open-domain QA tasks.

**TIP Exam Tip: REALM key: shift from implicit knowledge (in weights) to explicit knowledge (via retrieval). Instead of memorizing, model retrieves from Wikipedia. Benefits: no size increase needed, easy to update knowledge, better on knowledge-intensive tasks. Intuition: BERT=memorize, REALM=search.**

### LINK Bridge to Next Concept

REALM retrieves information and extracts short answer spans - similar to reading comprehension. But what if we need full, natural, detailed answers? **RAG (Retrieval-Augmented Generation)** extends REALM by adding a powerful generative component that can produce fluent, detailed answers using the retrieved documents.

# RAG

Retrieval-Augmented Generation - Search + Generate

RAG combines the best of both worlds: the **search capability of REALM** and the **generative power of Seq2Seq models**. Unlike REALM which extracts answer spans, RAG produces full, natural, detailed answers. The pipeline: Input query - Query Encoder encodes it - Retriever finds top-K relevant documents - Generator (Seq2Seq) reads query + documents - produces final answer. REALM = retrieve + extract. RAG = retrieve + **generate**.

## \* RAG Architecture - 3 Main Components

### ■ 1. Query Encoder

Encodes input query  $x$  into a dense vector  $q(x)$ .

### ■ 2. Retriever (Non-parametric)

Uses **MIPS (Maximum Inner Product Search)** to find top-K relevant documents  $z_1, z_2, \dots, z_K$ .

### ■ 3. Generator (Parametric)

Seq2Seq model (e.g., BART). Produces final output  $y$  using both query AND retrieved documents.

### ■ Memory types

Parametric memory: stored in generator weights. Non-parametric memory: the external document index.

### ■ Training

Entire system trained **end-to-end**. Gradients flow through both retriever and generator.

## > Marginalization

### ■ What

Final prediction computed by combining outputs from **multiple retrieved documents**.

### ■ Why

Reduces dependency on a single document being perfectly relevant.

### ■ How

Generate answer for each document - marginalize (combine) all predictions.

### ■ Benefit

More robust answers even if no single document has the full answer.

## = RAG vs REALM

### ■ REALM focus

Short answer extraction.

### ■ RAG focus

**Full answer generation.**

### ■ REALM output

Answer span from document.

### ■ RAG output

Generated natural language answer.

### ■ Key advantage

RAG produces more detailed, natural, and flexible answers.

## ! Why RAG is Powerful

### ■ Key advantage

Combines **search (retrieval)** + **reasoning (generation)**.

### ■ Better than pure LMs

Doesn't need to memorize - retrieves up-to-date info.

### ■ Better than pure QA

Generates natural full answers, not just extracted spans.

### ■ Scalable

Retrieval index can be updated without retraining the model.

### ■ Applications

Open-domain QA, Fact verification, Dialogue systems, Question Generation.

**TIP Exam Tip:** RAG = Query Encoder + Retriever (MIPS, top-K docs) + Generator (Seq2Seq/BART).  
**Pipeline:** input - encode - retrieve top-K - generate answer using query + docs. **Marginalization:** combine answers from multiple docs. RAG = retrieve + generate (vs REALM = retrieve + extract).

## LINK Bridge to Next Concept

We've covered encoder-based models (BERT) and retrieval models (REALM, RAG). Now let's step back and understand the three major pretraining architectures: **Encoders, Decoders, and Encoder-Decoders** - each has a different objective and best use case.



# Architectures and Decoder Models

GPT, T5, BART and the Decoder-only Family

1. Three Pretraining Architectures
2. T5 - Text-to-Text Transfer Transformer
3. BART - Denoising Sequence-to-Sequence
4. Pretraining Decoders
5. GPT-1 - First Decoder-only Pretrained Model
6. GPT-2 - Scaling + Zero-Shot Emergence
7. GPT-3 + In-Context Learning

# Three Pretraining Architectures

Encoders, Decoders, Encoder-Decoders - Each Has a Purpose

The architecture of a model determines **how we pretrain it** and **what tasks it's best for**. Encoders (like BERT) see the full input - great for understanding. Decoders (like GPT) only see past tokens - great for generation. Encoder-Decoders (like T5, BART) combine both - great for tasks requiring understanding AND generation. **Each architecture needs a different pretraining objective.**

## = The Three Types

### ■ 1. Encoders (BERT)

**Bidirectional context** (left + right). Pretraining: Masked Language Modeling (MLM). Good for: understanding tasks - classification, NER, QA (extractive).

### ■ 2. Decoders (GPT)

**Left-to-right (past only)**. Pretraining: Next word prediction (language modeling). Good for: text generation, dialogue, completion.

### ■ 3. Encoder-Decoder (T5/BART)

**Combines both**. Encoder: full bidirectional input. Decoder: autoregressive output. Good for: translation, summarization, QA, any task needing understanding + generation.

### ■ Key insight

Each architecture needs a **different pretraining objective**.

### ■ Conclusion

Each architecture is best suited for different types of tasks - no one-size-fits-all.

## @ Pretraining Encoder-Decoder Models

### ■ Key idea

Use a **modified language modeling objective**.

### ■ Encoder role

Takes **full input** - learns bidirectional representation.

### ■ Decoder role

Predicts future tokens - like language modeling (left-to-right).

### ■ Important insight

Input is **given to encoder** (not predicted). Decoder generates output using encoder information.

### ■ Combines

BERT-like understanding (encoder) + GPT-like generation (decoder).

### ■ Example models

T5, BART - both use encoder-decoder architecture.

**TIP Exam Tip: Three architectures: Encoders (BERT) = bidirectional, MLM, understanding tasks. Decoders (GPT) = left-to-right, next word prediction, generation. Encoder-Decoder (T5/BART) = combines both, understanding + generation tasks.**

### LINK Bridge to Next Concept

The most famous encoder-decoder model is **T5**, which reformulates EVERY NLP task as a text-to-text problem - one unified framework for all tasks. This was a conceptual breakthrough in model design.

# T5 - Text-to-Text Transfer Transformer

Every NLP Task as Text to Text

T5 proposes a beautifully simple idea: **convert every NLP task into a text-to-text format**. Translation? Input: 'translate English to German: I love you.' Output: 'Ich liebe dich.' Classification? Input: 'cola sentence: The dog barked.' Output: 'acceptable.' QA? Input: 'question: ... context: ...' Output: 'answer.' This unified framework means ONE architecture can handle ALL tasks - no task-specific heads needed.

| * T5 Core Concept                                                                                                                                                                                                                                                                                                                                                                                                                 | ~ Examples of Text-to-Text                                                                                                                                                                                                                                                                                                                                                                                                                  |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <ul style="list-style-type: none"> <li>■ <b>Full form</b><br/>Text-to-Text Transfer Transformer</li> <li>■ <b>Proposed by</b><br/>Colin Raffel et al., 2019</li> <li>■ <b>Core idea</b><br/>Every task - <b>Input text - Output text</b></li> <li>■ <b>Architecture</b><br/>Transformer <b>Encoder-Decoder</b></li> <li>■ <b>Dataset</b><br/><b>C4</b> (Colossal Clean Crawled Corpus) - large-scale cleaned web text.</li> </ul> | <ul style="list-style-type: none"> <li>■ <b>Translation</b><br/>'translate English to German: ...' - translated sentence</li> <li>■ <b>Classification</b><br/>'cola sentence: ...' - 'acceptable / not acceptable'</li> <li>■ <b>QA</b><br/>'question: ... context: ...' - 'answer'</li> <li>■ <b>Summarization</b><br/>'summarize: ...' - summary</li> <li>■ <b>Key</b><br/><b>Task type embedded in the input text itself.</b></li> </ul> |

## ! Span Corruption Pretraining (VERY IMPORTANT)

### ■ What

T5 uses **Span Corruption (denoising)** as pretraining objective.

### ■ Step 1

Remove spans (continuous chunks) from text.

### ■ Step 2

Replace them with special tokens ( , ).

### ■ Step 3

Model predicts the missing spans.

### ■ Example input

'Thank me to my party week.'

### ■ Example target

' for inviting last '

### ■ Why span corruption?

More efficient than token masking (BERT) - predicts only missing spans, not full sequence.

## @ Key Advantages of T5

### ■ Unified framework

Single framework for ALL NLP tasks - no task-specific architecture.

### ■ No task-specific heads

Task encoded in input text itself.

### ■ Combines strengths

BERT-like understanding (encoder) + GPT-like generation (decoder).

### ■ Works for

Classification (GLUE), QA (SQuAD), Summarization (CNN/DM), Translation.

### ■ Exam point

T5 uses encoder-decoder. Reformulates ALL tasks to text-to-text. Trained on C4. Uses span corruption.

**TIP Exam Tip: T5 = Text-to-Text Transfer Transformer. Every task - input text - output text.**

**Architecture: encoder-decoder. Dataset: C4. Pretraining: span corruption (remove spans - model predicts). Unified framework: no task-specific architecture needed.**

## LINK Bridge to Next Concept

T5 uses span corruption. BART uses a more general **denoising** approach - it can corrupt text in many different ways (masking, deletion, permutation, infilling) and learns to reconstruct the original. This flexibility makes BART robust and versatile.

# BART

Bidirectional + Auto-Regressive Transformers - Denoising Seq2Seq

BART combines the strengths of BERT and GPT into one model. The core idea: **corrupt input text in various ways, then train the model to reconstruct the original**. The Encoder reads the corrupted input with full bidirectional attention (like BERT). The Decoder generates the original text left-to-right (like GPT). This denoising training makes BART robust and excellent at both understanding AND generation.

## \* BART Overview

### ■ Full form

Bidirectional and Auto-Regressive Transformers

### ■ Proposed by

Mike Lewis et al., 2019

### ■ What is BART?

Denoising Sequence-to-Sequence Transformer.

### ■ Core idea

Corrupt input text - Reconstruct original text.

### ■ Pipeline

Noisy input - Encoder - Decoder - Original text

## = Architecture

### ■ Encoder

Full **bidirectional context** (BERT-like) - reads corrupted input.

### ■ Decoder

Generates tokens **left-to-right** (GPT-like) - produces clean output.

### ■ Pretraining objective

**Denoising (Reconstruction Task)**

### ■ Key insight

Encoder processes noisy input. Decoder reconstructs clean output.

## ~ Types of Noise Used (Corruption Techniques)

### ■ Token masking

Replace random tokens with [MASK] (like BERT).

### ■ Token deletion

**Delete** random tokens entirely from input.

### ■ Text infilling

Mask entire spans with a single [MASK] token.

### ■ Sentence permutation

**Shuffle** sentence order - model must restore order.

### ■ Document rotation

Rotate document to start from a random token - model identifies original start.

### ■ Key strength

**Flexible noise - model becomes robust** - learns long-range dependencies.

| Model | Type            | Strength      |
|-------|-----------------|---------------|
| BERT  | Encoder         | Understanding |
| GPT   | Decoder         | Generation    |
| BART  | Encoder-Decoder | Both          |

**TIP Exam Tip: BART = Denoising Seq2Seq Transformer. Core: corrupt input - reconstruct original. Encoder: bidirectional (BERT-like). Decoder: left-to-right (GPT-like). Noise types: token masking, token deletion, text infilling, sentence permutation, document rotation. One-line: BART combines BERT's understanding and GPT's generation.**

### LINK Bridge to Next Concept

We've seen encoder-decoder models. Now let's focus on **decoder-only models** - how they're pretrained and how they handle classification tasks without an encoder. This sets up GPT-1, the first major decoder-only pretrained model.

# Pretraining Decoders

How Decoder-Only Models Are Pretrained and Fine-tuned

Decoder-only models are pretrained as **language models** - they learn to predict the next word using only past context (left-to-right, no bidirectional attention). For classification tasks, a simple trick is used: take the **last hidden state** and pass it through a new linear layer. This works because the last token's hidden state has 'seen' the entire sequence through the decoder's attention mechanism.

## = The Three Architecture Types - Summary

### ■ 1. Encoders (BERT)

Bidirectional context (left + right). Strong for understanding. Used for: classification, NER, QA (extractive).

### ■ 2. Decoders (GPT)

Autoregressive:  $p(w_t | w_1, \dots, w_{t-1})$ . Only past context. Best for: text generation. Limitation: cannot see future tokens.

### ■ 3. Encoder-Decoder (T5, BART)

Encoder understands input. Decoder generates output. Best for: translation, summarization.

## ! Method: Classification Fine-tuning for Decoders

### ■ Step 1

Input - Decoder - hidden states  $h_1, h_2, \dots, h_T$

### ■ Step 2

Take last hidden state  $h_T$  (summary of entire sequence).

### ■ Step 3

Apply new linear layer:  $y = Ah_T + b$

### ■ Key point 1

Linear layer is **new (not pretrained)** - only this layer is randomly initialized.

### ■ Key point 2

**Entire model is fine-tuned** - pretrained weights updated.

### ■ Key point 3

$h_T$  acts as **sentence representation** (like [CLS] in BERT).

| Feature      | Classification    | Generation              |
|--------------|-------------------|-------------------------|
| Output       | Label             | Sequence                |
| Uses         | Last hidden state | All steps               |
| Linear layer | New (added)       | Pretrained              |
| Use case     | Sentiment, QA     | Dialogue, summarization |



**TIP Exam Tip:** Decoders pretrained as language models (predict next word). For classification fine-tuning: input - decoder - take last hidden state  $h_T$  - apply new linear layer  $y = Ah_T + b$ . Key: linear layer is new (not pretrained),  $h_T$  acts as sentence representation.

### LINK Bridge to Next Concept

Now let's meet the most famous decoder-only models. **GPT-1** was the first to show that pretraining a large decoder model and then fine-tuning it works extremely well across many tasks. This launched the GPT revolution.

# GPT-1

## The First Generative Pretrained Transformer

GPT-1 was the first major demonstration that **pretraining a large decoder-only Transformer on language modeling, then fine-tuning on specific tasks**, works remarkably well. It established the decoder-only pretrain + fine-tune paradigm. The clever fine-tuning trick: convert all tasks to a single sequence format and use special tokens - no architectural changes needed per task.

### \* GPT-1 Overview

#### ■ Full form

Generative Pretrained Transformer

#### ■ Authors

Radford et al., 2018 (OpenAI)

#### ■ Architecture

Only **Transformer Decoder** - no encoder.

#### ■ Training data

BooksCorpus (~7000 books) - long text - learns long-range dependencies.

#### ■ Tokenization

Byte Pair Encoding (BPE) - ~40k tokens.

### = Model Details

#### ■ Layers

12 transformer layers

#### ■ Hidden size

768

#### ■ Feedforward

3072

#### ■ Parameters

**117M parameters**

#### ■ Key

Decoder only - autoregressive language modeling.

### ! GPT Fine-Tuning Trick (Input Formatting)

#### ■ Core idea

Convert tasks to **single sequence format** - no architecture changes.

#### ■ Format

[START] Premise [DELIM] Hypothesis [EXTRACT]

#### ■ Prediction

Use representation of **[EXTRACT]** token for task prediction.

#### ■ Similar to

BERT's [CLS] token - captures task-level information.

#### ■ Benefit

Same model, same architecture - works for many tasks by changing input format.

#### ■ Performance

Works well on MNLI, SNLI, QNLI, RTE benchmarks.

**TIP Exam Tip: GPT-1: Generative Pretrained Transformer. Decoder-only, 117M params, BPE tokenization. Pretrained on BooksCorpus (next word prediction). Fine-tuning trick: [START] Premise [DELIM] Hypothesis [EXTRACT]. Use [EXTRACT] token for prediction (similar to BERT's [CLS]).**

### LINK Bridge to Next Concept

GPT-1 showed the decoder paradigm works. GPT-2 scaled it up massively - more data, larger model - and discovered something unexpected: large language models can perform tasks **without any fine-tuning at all**. This is **Zero-Shot learning**.

# GPT-2 - Scaling + Zero-Shot

More Data + Bigger Model = Emergent Zero-Shot Ability

GPT-2 scaled GPT-1: **more data (internet text) + much larger model (1.5B parameters)**. The key discovery: at sufficient scale, the model starts performing tasks **without any task-specific fine-tuning** - just from the input prompt. This is **Zero-Shot learning** - the model generalizes from pretraining alone. Key statement: 'Language Models are Unsupervised Multitask Learners.'

## > GPT-2 Improvements

### ■ Scale

Model size: 117M - **1.5B parameters**

### ■ Data

Training data: 4GB - **40GB** (internet text - WebText)

### ■ Result

Generates **human-like text** with strong coherence.

### ■ Key takeaway

**Decoder models are powerful text generators.**

### ■ Scaling

Bigger models - better performance.

## ! Zero-Shot Learning

### ■ Definition

Perform tasks **without examples or fine-tuning** - only instruction.

### ■ How

Tasks converted to sequence prediction problems: 'predict the next word/token'

### ■ Key idea

Model learns multiple tasks implicitly during pretraining from raw text.

### ■ GPT-2 finding

GPT-2 performs well on multiple tasks with no task-specific fine-tuning.

### ■ Example

QA: input = passage + question - answer predicted as next tokens.

## @ GPT-2 Emergent Zero-Shot Performance

### ■ Zero-shot prompting trick

Add simple cues: 'TL;DR' for summarization - helps model understand task.

### ■ Result

Model generates summaries without any training examples.

### ■ Performance

Below supervised but still useful for many tasks.

### ■ LAMBADA task

Requires understanding long context - larger models perform better.

### ■ Key observation

GPT-2 generalizes to many tasks - no task-specific fine-tuning used.

### ■ Important statement

Language Models are Unsupervised Multitask Learners

**TIP Exam Tip: GPT-2: 1.5B params, 40GB data. Introduces zero-shot learning - perform tasks without fine-tuning. Key statement: 'Language Models are Unsupervised Multitask Learners.'** Scaling (more data + bigger model) dramatically improves performance.

## LINK Bridge to Next Concept

GPT-2 showed zero-shot learning. GPT-3 took this much further with 175B parameters and introduced **In-Context Learning** - the model can learn new tasks just from examples provided in the prompt. No gradient updates. No fine-tuning. Just prompting.

# GPT-3 + In-Context Learning

175B Parameters - Learning from Examples in the Prompt

GPT-3 is a revolution. At **175 billion parameters** - 100x larger than GPT-2 - it demonstrates **In-Context Learning (ICL)**: the model can perform new tasks by being shown a few examples in the prompt, **without any gradient updates or fine-tuning**. The prompt acts as a mini training dataset that the model can 'read' and follow. This is possible because during pretraining, GPT-3 has seen enough diverse text to recognize and follow any task pattern.

## ^ GPT-3 Scale

- **Parameters**  
175 billion parameters
- **Scale**  
Much larger than GPT-2 (1.5B).
- **New capability**  
In-Context Learning (ICL)
- **Key insight**  
Increasing model size - new capabilities emerge.
- **Training**  
Same as GPT-2 - next word prediction, just larger.

## \* In-Context Learning Definition

- **Definition**  
Model learns task from **examples in input**, without training.
- **No gradient update**  
No backpropagation. No weight updates.
- **No fine-tuning**  
No task-specific training.
- **Learning source**  
**Learns from prompt only.**
- **Key idea**  
Prompt = implicit task description + example dataset.

## ! Types of In-Context Learning

- **Zero-shot**  
Only **instruction** - no examples. Lowest performance.
- **One-shot**  
One **example** provided. Better performance.
- **Few-shot**  
Multiple **examples**. Best performance.
- **Example (few-shot)**  
hello - bonjour, thanks - merci, cat - chat, dog - ? Output: chien
- **Key insight**  
More examples in context - better results.

## @ Why In-Context Learning Works

### ■ Pretraining insight

Model has already learned patterns from diverse pretraining text.

### ■ Prompt acts like

Mini training dataset for the task.

### ■ Implicit task

Prompt = task instruction + example input-output pairs.

### ■ No update needed

Model recognizes pattern and applies it - no learning needed.

### ■ Important statement

Language Models are Few-Shot Learners

**TIP Exam Tip: GPT-3: 175B params. Key: In-Context Learning - model learns from examples in prompt, NO gradient update, NO fine-tuning. Types: Zero-shot (only instruction), One-shot (one example), Few-shot (multiple examples). Why it works: pretraining taught diverse patterns, prompt = mini dataset.**

## LINK Bridge to Next Concept

GPT-3's in-context learning is impressive - but these are **emergent abilities** that only appear at large scale. The model also needs to be made **useful and aligned** with human preferences. Raw GPT-3 just continues text patterns - it doesn't understand user intent. This is where **Post-Training** comes in.

# Post-Training and Alignment

Making Models Helpful, Harmless, and Honest

1. Post-Training Overview
2. RLHF - Reinforcement Learning from Human Feedback
3. DPO - Direct Preference Optimization
4. Emergent Abilities and Scaling Laws
5. Language Models as World Models



# Post-Training Overview

From Raw LLM to Helpful Assistant

After pre-training, the model is essentially a sophisticated text-continuation machine - **raw LLMs don't understand user intent, they just continue patterns**. Post-training is the process of **improving the model's behavior, usefulness, and alignment** after pre-training is complete. Goal: make the model give **helpful, correct, and safe answers** that satisfy human preferences - not just grammatically correct text continuations.

## \* What is Post-Training?

### ■ When

Happens **after pre-training** is complete.

### ■ Focus

Improving behavior, usefulness, and alignment.

### ■ Goal

Make model give **helpful, correct, safe answers**.

### ■ Raw LLM problem

Doesn't understand user intent - just continues text patterns.

### ■ One-line

We transformed basic language models into assistants using post-training.

## = Pretraining vs Post-training

### ■ Pre-training

Learns **language patterns** - grammar, facts, world knowledge.

### ■ Post-training

Learns **how to respond properly** - helpful, safe, aligned.

### ■ Pre-training data

Massive unlabeled internet text.

### ■ Post-training data

Human feedback, instruction-output pairs, preference data.

## ! Key Post-Training Techniques

### ■ 1. RLHF

Reinforcement Learning from Human Feedback - train model to maximize human reward.

### ■ 2. DPO

Direct Preference Optimization - simpler alternative to RLHF using pairwise comparisons.

### ■ 3. Instruction Tuning

Train on (instruction, output) pairs - model learns to follow instructions.

### ■ 4. In-Context Learning (ICL)

No weight update. Learns from examples in prompt. Works via pattern recognition.

### ■ Final conclusion

Language models are not aligned with user intent - post-training is the solution.

**TIP Exam Tip:** Post-training: happens after pretraining. Goal: helpful, correct, safe responses. Difference: pretraining = language patterns, post-training = how to respond properly. Key techniques: RLHF, DPO, Instruction tuning.

### LINK Bridge to Next Concept

The most powerful post-training technique is **RLHF - Reinforcement Learning from Human Feedback**. Humans can't specify exactly what a 'good' answer looks like in mathematical terms, so we use their comparative judgments to train a reward signal.

# RLHF

## Reinforcement Learning from Human Feedback

RLHF solves a fundamental problem: we want the model to satisfy human preferences, but **human reward is not differentiable** - we can't directly backpropagate through human judgments. The solution: cast the problem as Reinforcement Learning. The language model is the **Agent**, generated text is the **Action**, and human feedback is the **Reward signal**. Good outputs increase in probability; bad outputs decrease.

### ? Why RL?

#### ■ Human reward

**Not differentiable** - can't backpropagate through human judgment.

#### ■ Not available

Can't get human feedback for every possible output.

#### ■ Standard supervised

Cannot handle this well - needs exact labels.

#### ■ RL solution

LM generates sequentially (like actions). Each output evaluable as reward signal.

#### ■ Goal aligns

**Maximize cumulative reward** = maximize human satisfaction.

### = RL Framework for LMs

#### ■ Agent

Language Model

#### ■ Action

Generated text (output)

#### ■ Reward

Human feedback score

#### ■ Objective

Maximize expected human reward:  $E[R(s)]$

#### ■ Why RL fits

LM outputs are sequential - like actions. Each output can be evaluated.

## ! Learning Process

### ■ Step 1

Generate output from language model.

### ■ Step 2

Human rates the output (or a trained reward model does it).

### ■ Step 3

**Adjust probabilities accordingly:**

#### ■ If output gets HIGH reward

Increase its probability - make model more likely to produce it.

#### ■ If output gets LOW reward

Decrease its probability - make model less likely to produce it.

### ■ Result

Model learns to generate outputs humans prefer.

**TIP Exam Tip: RLHF: human reward not differentiable - use RL. Framework: Agent=LM, Action=text, Reward=human feedback. Learning: generate - get reward - adjust probabilities (high reward - increase prob, low - decrease). Objective: maximize expected human reward  $E[R(s)]$ .**

## LINK Bridge to Next Concept

RLHF requires training a separate reward model and is complex to implement and stabilize. **DPO (Direct Preference Optimization)** is a simpler alternative that achieves similar results using pairwise comparisons - 'Answer A is better than Answer B'.

# DPO + Emergent Abilities

Simpler Alignment + Capabilities That Emerge at Scale

DPO is a simpler alternative to RLHF that doesn't require a separate reward model. Meanwhile, a fundamental phenomenon in large language models is **Emergent Abilities**: capabilities that appear suddenly and unexpectedly as models scale up - GPT-1 had none, GPT-2 showed zero-shot, GPT-3 showed few-shot learning. These abilities are not explicitly trained for - they just **emerge** from scale.

## = DPO - Direct Preference Optimization

### ■ Alternative to

RLHF - simpler and more stable.

### ■ Method

Uses **pairwise comparison**: 'Answer A is better than B.'

### ■ No reward model

Doesn't need a separate trained reward model.

### ■ More stable

Simpler loss function - more stable training than RL.

### ■ One-line

Post-training = refining model behavior to make it useful and aligned.

## ! Emergent Abilities - What They Are

### ■ Definition

Capabilities that appear **without explicit training** - emerge from scale.

### ■ GPT-1 (117M)

Basic language generation.

### ■ GPT-2 (1.5B)

**Zero-shot learning** emerges.

### ■ GPT-3 (175B)

**Few-shot (In-Context) learning** emerges.

### ■ Meaning of emergent

Ability appears **only after model reaches large scale**.

## @ Scaling Laws - Why Bigger Models Are Better

### ■ Key insight

Scaling (more data + bigger model) **dramatically improves performance.**

### ■ Important statement

Language Models are Few-Shot Learners - GPT-3 paper.

### ■ Important statement 2

Language Models are Unsupervised Multitask Learners - GPT-2 paper.

### ■ Small models

Poor few-shot performance

### ■ Large models

Strong few-shot capability

### ■ Few-shot as emergent

Few-shot ability improves with model size - classic emergent property.

### ■ Exam point

Pretraining - general intelligence. Finetuning - task specialization. Scaling - new abilities.

**TIP Exam Tip: DPO = simpler alternative to RLHF. Uses pairwise comparison (A better than B). More stable, no reward model needed. Emergent abilities: capabilities appearing only at large scale. GPT-1 basic, GPT-2 zero-shot, GPT-3 few-shot. Key: scaling = new emergent capabilities.**

## LINK Bridge to Next Concept

Large language models don't just generate text - they build internal representations of how the world works. They model **Agents, Beliefs, and Actions**. This leads to an interesting question: are language models actually modeling the world?

# LMs as World Models + Scaling Fine-tuning

Internal World Representations + Multi-Task Fine-tuning

Language models don't just learn language - they build **internal representations of how the world works**. They learn about Agents (who exists), Beliefs (what they know), and Actions (what they can do). At the same time, fine-tuning has also scaled: instead of fine-tuning one model per task, we now fine-tune ONE model on **many tasks simultaneously** - creating general-purpose assistants.

## @ LMs as World Models

### ■ Core idea

Language models **simulate understanding of the real world**.

### ■ They model

**Agents** (people), **Beliefs** (what they know), **Actions** (what they do).

### ■ Example (Pat scenario)

Same situation - different predictions based on Pat's knowledge.

### ■ Implication

Understanding depends on **context + knowledge**.

### ■ Key concept

World Modeling - LLM builds internal representation of situations using patterns.

## > Scaling Up Fine-tuning

### ■ Traditional approach

One model - one task.

### ■ Modern approach

One model - many tasks simultaneously.

### ■ Benefit

Single model can: summarize, translate, answer questions, generate text.

### ■ Key idea

**Multi-task fine-tuning** - makes models general-purpose assistants.

### ■ Example

FLAN-T5 fine-tuned on 1800+ tasks simultaneously.

## ! One-Line Summaries (Exam-Ready)

### ■ Raw LLMs

Don't understand user intent - just continue patterns.

### ■ Post-training

Transforms raw LLMs into helpful assistants.

### ■ Pretraining

- general intelligence (language patterns, facts, reasoning).

### ■ Fine-tuning

- task-specific specialization.

### ■ Scaling

- emergent abilities (zero-shot, few-shot).

### ■ Post-training

- alignment (helpful, safe, correct responses).

**TIP Exam Tip: LMs as world models: represent Agents, Beliefs, Actions - simulate world understanding. Scaling fine-tuning: one model fine-tuned on many tasks simultaneously = general-purpose assistant. Traditional: one model one task. Modern: one model many tasks.**

## LINK Bridge to Next Concept

Now we understand how models are trained and aligned. A key question is: **how do we give instructions to these models without fine-tuning?** The answer is **Prompting** - converting any task into a text completion problem that the model can solve using its pretrained knowledge.



# Prompting and Instruction Tuning

Eliciting Knowledge Without Training

1. Prompting - Formal Definition and Workflow
2. Chain-of-Thought (CoT) Prompting
3. Instruction Fine-tuning
4. Limitations of Instruction Fine-tuning + MMLU
5. Probing - Analyzing What Models Know

# Prompting

Converting Tasks into Text Completion - No Training Required

Prompting reformulates any NLP task into a **text completion problem** that the language model was already trained to solve. Instead of retraining or fine-tuning, we convert the task into a text template and let the model complete it. A prompt is 'a task reformulated as text that looks like what the language model saw during pretraining.' The model predicts missing parts using its learned patterns - no gradient updates, no training.

## ! Prompting Workflow - 3 Steps

### ■ Step 1: Prompt Addition

Add input into a **template** that creates a fill-in-the-blank problem. Example: 'I love this movie.' - 'I love this movie. Overall, it was a [Z] movie.'

### ■ Step 2: Answer Prediction

Model predicts the missing part [Z] using context + learned patterns. Output is **free text** (not fixed labels). Example: 'fantastic'

### ■ Step 3: Answer Mapping

Convert model output (z) to final answer (y). Example: 'fantastic' - Positive sentiment.

### ■ Key idea

Input - Prompt - Prediction - Output. **No model training required.**

### ■ Works by

Reframing task as text completion - exactly what model was trained to do.

## = Prompting vs Fine-tuning

### ■ Prompting

**Learning at inference time.** No training. Use prompt examples.

### ■ Fine-tuning

**Learning during training.** Gradient updates. Labeled data needed.

### ■ Key insight

Prompting can replace fine-tuning for many tasks.

### ■ Advantage

No training cost. Instantly adaptable. Works with any pretrained LM.

## ~ Example Tasks via Prompting

### ■ Sentiment

Input: 'I love this movie.' - Prompt adds: 'Overall, it was a [Z] movie.' - Output: 'fantastic' - Positive.

### ■ Translation

Prompt: 'English: I love this movie. French: [Z]' - Output: 'J'adore ce film.'

### ■ Key

Task expressed as **text completion**.

**TIP Exam Tip: Prompting = reformulate task as text completion. No training. Workflow: (1) Prompt Addition: add input to template with [Z] slot. (2) Answer Prediction: model predicts [Z]. (3) Answer Mapping: convert [Z] to final answer. Key: Input - Template - Prediction - Output.**

### LINK Bridge to Next Concept

Standard prompting gives direct answers but struggles with complex reasoning tasks. The solution: **Chain-of-Thought (CoT) prompting** - instead of asking for the answer directly, ask the model to reason step by step first. This dramatically improves performance on hard tasks.

# Chain-of-Thought (CoT) Prompting

## Step-by-Step Reasoning for Complex Tasks

Standard prompting gives a direct answer - often wrong for multi-step reasoning tasks. **CoT prompting asks the model to generate step-by-step reasoning before giving the answer.** Instead of asking 'What is the answer?', we ask 'Let's think step by step: ...' This mirrors how humans solve complex problems - think through it, then answer. CoT is an **emergent property**: it works well only in large language models.

### = Standard vs CoT Prompting

#### ■ Standard prompt

Model gives **direct answer**.

#### ■ Standard result

Often incorrect for reasoning tasks.

#### ■ CoT prompt

Model generates **step-by-step reasoning** then answers.

#### ■ CoT result

**Significantly improves accuracy** on complex tasks.

#### ■ Key idea

Reasoning steps - better results.

### ! CoT Performance Numbers

#### ■ Normal zero-shot

~17% accuracy on hard reasoning tasks.

#### ■ Zero-shot CoT

+78% improvement! Massive gain.

#### ■ Few-shot CoT

Even higher - best performance.

#### ■ Insight

CoT **dramatically improves** reasoning task accuracy.

## @ Types of CoT Prompting

### ■ Standard CoT

Provide few-shot examples WITH reasoning steps. Example: 'Q: ... Step 1: ... Step 2: ... Answer: ...'  
Traditional CoT requires writing examples.

### ■ Zero-shot CoT

**No examples needed.** Just add trigger phrase: 'Let's think step by step.' Model automatically generates reasoning.

### ■ Effect of wording

'Let's think step by step' - high accuracy. 'Let's work this out step by step' - even better. Weak prompts - low performance.

### ■ Limitation

Writing few-shot reasoning examples is time-consuming.

### ■ CoT as emergent

CoT works better as model size increases. Small models - little benefit. Large models - huge improvement.

### ■ CoT insight

Simple prompts can **activate reasoning ability** already in the model.

**TIP Exam Tip:** CoT prompting = ask model to generate step-by-step reasoning before answering. Zero-shot CoT: just add 'Let's think step by step.' - no examples needed. Performance: normal zero-shot ~17%, zero-shot CoT +78% improvement. CoT is emergent - works mainly in large language models.

## LINK Bridge to Next Concept

Prompting is powerful but has limitations for complex tasks requiring precise knowledge. **Instruction Fine-tuning** trains models on thousands of diverse tasks using (instruction, output) pairs, making them generalize better to new instructions.

# Instruction Fine-tuning + MMLU

Training to Follow Instructions + Measuring Knowledge

Instruction fine-tuning trains a language model using **(instruction, output) pairs** across many diverse tasks. The model learns to follow instructions correctly and generate proper outputs. Before instruction fine-tuning, models generate multiple sentences and don't answer questions properly. After, they understand task instructions and give correct, targeted answers. MMLU benchmark measures this across 57 diverse subjects.

## \* Instruction Fine-tuning

### ■ What

Train model using **(instruction, output) pairs** across many tasks.

### ■ Model learns

Follow instructions. Generate correct outputs.

### ■ Process

Collect instruction-answer examples - fine-tune - evaluate on unseen tasks.

### ■ Key benefit

Model generalizes to **new tasks/questions** using learned reasoning patterns.

### ■ Dataset

Super-NaturalInstructions: 1600+ tasks, 3M+ examples.

## = Performance and Examples

### ■ FLAN-T5

T5 fine-tuned on ~1800 instruction-based tasks.

### ■ Bigger model

**Bigger model - bigger performance gain** from instruction tuning.

### ■ Self-instruct

Generate instruction data using strong LMs (Alpaca: LLaMA 7B + 52K examples).

### ■ Data quality

Less data + high quality can still work well.

### ■ Key insight

Instruction fine-tuning enables multi-task learning and general-purpose models.

## ! MMLU Benchmark

### ■ Full form

Massive Multitask Language Understanding

### ■ Contents

57 diverse subjects: Math, Physics, Biology, CS, Medicine, History, Law, etc.

### ■ Focus

Knowledge-intensive tasks requiring reasoning and domain knowledge.

### ■ Measures

General knowledge, Reasoning ability, Multi-domain understanding.

### ■ Model performance

GPT-3 and UnifiedQA perform better than random - but varies widely by subject.

### ■ Key insight

MMLU tests whether instruction fine-tuning truly improves generalization across domains.

**TIP Exam Tip: Instruction fine-tuning: train on (instruction, output) pairs across many tasks. Model learns to follow instructions and generalize to new tasks. Bigger model = bigger gain. MMLU: 57 subjects, knowledge-intensive, measures general knowledge + reasoning + multi-domain understanding.**

## LINK Bridge to Next Concept

Instruction fine-tuning has important limitations. Understanding these helps answer exam questions about WHY we need further alignment methods like RLHF.

# Limitations of Instruction Fine-tuning

## Why Instruction Tuning Is Not Enough

Instruction fine-tuning dramatically improves models but has fundamental limitations. These limitations motivate the need for RLHF and DPO - showing that simply training on instruction-output pairs is not sufficient to make a model truly aligned with what humans want.

### ! Four Key Limitations

#### ■ 1. Expensive

Collecting high-quality (input, output) data is **costly** - requires expert annotators.

#### ■ 2. No single correct answer

Open-ended tasks (story writing, explanations) have **multiple valid outputs**. Hard to define ground truth.

#### ■ 3. Equal penalty problem

Model treats all token-level mistakes equally. But in reality, some errors are much more serious than others.

#### ■ 4. Objective Mismatch (VERY IMPORTANT)

Model objective: **predict next word**. Desired goal: **satisfy human preferences**. These are fundamentally different goals - leads to misalignment.

### ? Objective Mismatch - The Core Problem

#### ■ Model trains on

Predict next word - language modeling.

#### ■ Humans want

Responses that are helpful, safe, accurate, and pleasant.

#### ■ The gap

A grammatically perfect response may still be unhelpful, wrong, or harmful.

#### ■ Solution needed

Must optimize directly for human preferences - not just next-word prediction.

#### ■ Why RLHF/DPO

They directly optimize for human preference signals, not word-prediction loss.

**TIP Exam Tip: Instruction fine-tuning limitations: (1) Expensive data collection, (2) No single correct answer for open tasks, (3) Equal penalty for all mistakes, (4) Objective mismatch - predicts next word but humans want preferences satisfied. Most important: objective mismatch - motivates RLHF/DPO.**



### LINK Bridge to Next Concept

Finally, let's understand **Probing** - a technique to analyze what linguistic knowledge a pretrained model has actually learned, without fine-tuning the model itself. This is used in research to understand what's inside these 'black box' models.

# Probing

Supervised Analysis - What Linguistic Knowledge Does the Model Have?

Probing is a method to **analyze what linguistic knowledge a pretrained model has learned** without modifying the model itself. A simple classifier is trained on top of the frozen pretrained embeddings to predict linguistic properties (POS tags, syntax, named entities). If the small classifier succeeds, it means the information was **already encoded** in the pretrained representations. The key: the pretrained model is **FROZEN** - only the tiny probe is trained.

## = Probing Process

### ■ Definition

Training a simple classifier on top of pretrained embeddings.

### ■ Goal

Check what **linguistic knowledge** the model has learned.

### ■ Step 1

Input sentence - pass through pretrained model (BERT).

### ■ Step 2

Get **contextual embeddings** for each token.

### ■ Step 3

Train a **small probe classifier** on these embeddings.

### ■ Step 4

Predict linguistic property (e.g., POS tags, dependency relations).

## ! Key Points

### ■ Pretrained model

**FROZEN (not updated)** - only the probe is trained.

### ■ Only probe trained

The tiny probe classifier learns - model weights unchanged.

### ■ Purpose

**Measure knowledge, not learn new knowledge.**

### ■ What can be tested

POS tagging, Syntax (dependency relations), Named entities.

### ■ Success means

Information was already in the pretrained representations.

**TIP Exam Tip: Probing = train simple classifier on FROZEN pretrained embeddings to test linguistic knowledge. Process: sentence - pretrained model - contextual embeddings - small probe - predict POS/syntax. Key: pretrained model is FROZEN, only probe is trained. Purpose: measure knowledge, not learn.**

## LINK Bridge to Next Concept

Now for the mathematical foundation of RLHF - **REINFORCE (Policy Gradient)** is the algorithm that makes RL work for language models. The challenge: reward is not differentiable and outputs are discrete, so standard backpropagation doesn't work directly.

# Mathematical Foundations

The Math Behind RL Training of Language Models

## 1. REINFORCE - Policy Gradient Algorithm

# REINFORCE - Policy Gradient

The Math Behind RL for Language Models

REINFORCE solves a fundamental challenge in RL training of language models: **the reward  $R(s)$  is not differentiable** (it's a human judgment or score) and **the output is discrete text** (cannot backpropagate through discrete choices). Standard backpropagation doesn't work here. The clever trick: the **Log-derivative trick** converts the gradient of reward expectation into something we CAN compute and backpropagate through.

## ? The Problem - Why Normal Backprop Fails

### ■ Goal

Maximize  $E_{s \sim p_{\theta}(s)}[R(s)]$  - expected reward over generated outputs.

### ■ Problem 1

Reward  $R(s)$  is **not differentiable** - it's a human score or evaluation function.

### ■ Problem 2

Output  $s$  is **discrete text** - cannot differentiate through discrete choices.

### ■ Problem 3

Normal backpropagation requires differentiable operations - doesn't work here.

### ■ Need

A way to estimate gradient without direct differentiation through reward.

## ! The Solution - Log-Derivative Trick

### ■ Step 3 (Trick)

Use the **log-derivative trick**:  $\text{grad}(p_{\theta}(s)) = p_{\theta}(s) * \text{grad}_{\theta}(\log(p_{\theta}(s)))$

### ■ Why this helps

Instead of differentiating through reward, we differentiate through **log probability of output**.

### ■ Simple meaning

Multiply reward by gradient of log probability - get usable gradient signal.

### ■ Step 4 (Scale)

Use reward to scale how much we update the model.

### ■ Final form (Step 5)

$E[R(s) * \text{grad}(\log(p_{\theta}(s)))]$

### ■ Practical form

Sum over  $s$ :  $p_{\theta}(s) * R(s) * \text{grad}(\log(p_{\theta}(s)))$

## @ Intuition - What This Means Practically

### ■ Good output HIGH reward

Increase its probability - positive gradient update.

### ■ Bad output LOW reward

Decrease its probability - negative or small gradient update.

### ■ Simple meaning

Reward acts as a **scaling factor** for how much we update the model.

### ■ Key identity

$\text{grad\_theta}(p\_theta(s)) = p\_theta(s) * \text{grad\_theta}(\log(p\_theta(s)))$

### ■ Final gradient

$E[R(s) * \text{grad\_theta}(\log(p\_theta(s)))]$  - multiply reward by gradient of log probability.

### ■ No differentiable reward needed

The trick avoids ever needing to differentiate through  $R(s)$ .

**TIP Exam Tip: REINFORCE: goal = maximize  $E[R(s)]$ . Problem: reward not differentiable, output is discrete. Solution: log-derivative trick -  $\text{grad}(p\_theta(s)) = p\_theta(s) * \text{grad}(\log(p\_theta(s)))$ . Final form:  $E[R(s) * \text{grad}(\log(p\_theta(s)))]$ . Intuition: good output - increase prob, bad - decrease prob.**

## LINK Bridge to Next Concept

You have now covered ALL concepts in NLP Pretraining - from the foundations of pretraining and tokenization, through the full BERT family, retrieval models, decoder architectures, post-training alignment, prompting techniques, and the mathematical underpinning of RL. Good luck on your exam!